

Motion Detection App

System architecture with direction tracking

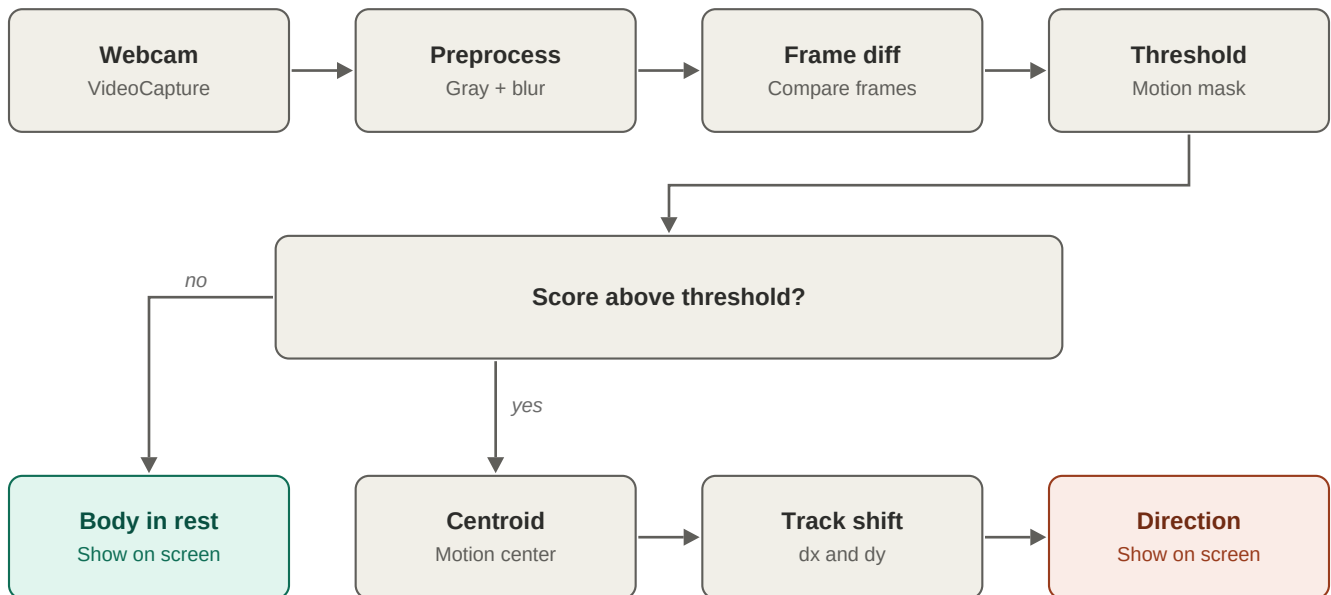
PYTHON / OPENCV / NUMPY

01 Overview

This application opens the webcam, analyses every frame, and reports what the camera sees in real time. It uses simple frame differencing, so it runs on an ordinary laptop with no machine-learning model.

- **Rest or motion:** the screen shows **Body in rest** when nothing moves, and **Motion detected** as soon as movement passes a threshold.
- **Direction:** while motion is detected, the app also reports whether the body or object is moving **left, right, up or down**.

02 Architecture



Gray boxes are processing steps. Teal is the resting output; coral is the motion output with direction.

03 Pipeline stages

Stage	What it does	Key call
1. Webcam capture	Reads frames from the default camera in a loop. The frame is mirrored so left and right match what the viewer sees.	VideoCapture(0), flip
2. Preprocess	Converts to grayscale and blurs to suppress sensor noise and lighting flicker.	cvtColor, GaussianBlur
3. Frame diff	Compares the current frame with the previous one to find the pixels that changed.	absdiff
4. Threshold	Turns the difference into a black and white motion mask. The white pixel count is the motion score.	threshold, countNonZero
5. Decision	A score above the limit means motion. Otherwise the body is at rest and direction history is cleared.	THRESHOLD
6. Centroid	Finds the center of the moving area from the moments of the mask.	moments
7. Track shift	Keeps the last 6 centroids and measures dx and dy from the oldest to the newest.	deque
8. Direction	The larger axis wins. Shifts inside the dead zone report no direction.	MIN_SHIFT

04 Direction logic

Image coordinates start at the top-left corner, so the vertical axis grows downward. A positive dy therefore means the motion is going **down** the screen.

Condition	Result
$ dx > dy $ and $dx > 0$	Right
$ dx > dy $ and $dx < 0$	Left
$ dx \leq dy $ and $dy > 0$	Down
$ dx \leq dy $ and $dy < 0$	Up
$\text{largest shift} \leq \text{MIN_SHIFT}$	Motion detected, no direction shown

05 Tuning parameters

Parameter	Starting value	Effect
THRESHOLD	1000 pixels (try 500 to 2000)	Higher values ignore small movements; lower values react to slight motion.
Pixel cutoff	25	Brightness change a pixel needs before it counts as changed.
Blur kernel	(21, 21)	A larger kernel removes more noise but also blurs small motion.
MIN_SHIFT	20 pixels (try 15 to 30)	Dead zone for direction. Larger values are steadier but slower to respond.
History length	6 frames	A longer window smooths noise but delays the direction output.
Rest delay (optional)	10 to 15 still frames	Wait this long before switching back to Body in rest to stop flicker.

06 Limitations and upgrades

- Frame differencing detects **any** movement, not only a human body. Add a person detector such as MediaPipe Pose or YOLO to react to people only.
- The centroid of a frame difference lies between the old and new positions, so direction is most reliable for steady, deliberate movement.
- Camera shake and automatic exposure changes create false motion. Keep the camera fixed.
- Dense optical flow (`calcOpticalFlowFarneback`) gives a motion vector directly and is a good upgrade for direction accuracy.

07 Implementation

Install the dependency with `pip install opencv-python`, save the script as `motion.py`, and run it. Press **q** in the video window to quit.

```
import cv2
from collections import deque

cap = cv2.VideoCapture(0)
prev = None
THRESHOLD = 1000      # changed pixels needed to count as motion
MIN_SHIFT = 20        # centroid movement (px) needed to report a direction
history = deque(maxlen=6)

while True:
    ok, frame = cap.read()
    if not ok:
        break
    frame = cv2.flip(frame, 1) # mirror view so left/right feel natural

    gray = cv2.GaussianBlur(cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY), (21, 21), 0)
    if prev is None:
        prev = gray
        continue

    diff = cv2.absdiff(prev, gray)
    mask = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)[1]
    score = cv2.countNonZero(mask)
    prev = gray

    m = cv2.moments(mask, binaryImage=True)
    if score > THRESHOLD and m["m00"] > 0:
        history.append((m["m10"] / m["m00"], m["m01"] / m["m00"]))
        direction = ""
        if len(history) == history.maxlen:
            dx = history[-1][0] - history[0][0]
            dy = history[-1][1] - history[0][1]
            if max(abs(dx), abs(dy)) > MIN_SHIFT:
                if abs(dx) > abs(dy):
                    direction = "Right" if dx > 0 else "Left"
                else:
                    direction = "Down" if dy > 0 else "Up"
            label = f"Motion detected {direction}.strip()"
        else:
            history.clear()
            label = "Body in rest"

    cv2.putText(frame, label, (20, 40), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
    cv2.imshow("Motion", frame)

    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cap.release()
cv2.destroyAllWindows()
```